Table des matières

Cours 06 — Orchestrer et rattraper 1
Objectifs 1
1. Ce qu'un orchestrateur fait, et ce qu'il ne fait pas 2
2. Le DAG, et ce que ses flèches ne disent pas 3
3. La planification, et l'intervalle de données 4
4. Le rattrapage 4
5. Rejouable : la fenêtre, le journal, et la reconstruction 5
6. Ce qui rate, ce qui tarde, ce qui attend 5
7. Ce qui se partage se réserve 6
8. Le point de couture 6
9. Un pipeline orchestré, en pratique 7
À retenir 7
Exercices 8
Annexe A — Le détail des étapes du labo 8

Cours 06 — Orchestrer et rattraper

Théorie · Cours 6/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → **ingestion** → lac → entrepôt → restitution. Le Cours 05 a fait du pipeline un programme qu'on lance d'une commande. Ce cours-ci décide quand cette commande part, sur quelle tranche de temps elle travaille, ce qui se passe quand elle rate, et ce qui arrive quand on la met en route avec trois mois de retard.

Objectifs

- Dire ce qu'un orchestrateur fait, ce qu'il ne fait pas, et nommer les services qui le composent.
- Lire un DAG : ses tâches, ses dépendances, et ce que ses flèches ne disent pas.
- Distinguer l'intervalle de données d'une exécution de l'heure à laquelle elle tourne.

© 2026 iAlexMG. Tous droits réservés.

- Expliquer le rattrapage, et ce qu'il exige des tâches pour donner un résultat juste.
- Reconnaître les mécaniques qui rendent un pipeline orchestré rejouable, et ce que chacune couvre.
- Choisir entre une nouvelle tentative, une alerte de retard et une réservation de ressource.
- Situer le point de couture entre ce cours et la formation Analytics Engineering.

---

1. Ce qu'un orchestrateur fait, et ce qu'il ne fait pas

Un orchestrateur de flux de données (*workflow orchestrator*) répond à quatre questions, et à quatre seulement.

La question	Ce qu'il en fait
Quoi lancer	il lit des définitions de graphes, et connaît chaque tâche par son nom
Quand	une expression de planification, et la date à partir de laquelle elle vaut
Dans quel ordre	les dépendances déclarées entre tâches, et rien d'autre
Ce qui s'est passé	l'état de chaque exécution et de chaque tâche, ses journaux, ses durées

- Il ne transforme aucune donnée, et ne connaît ni vos fichiers, ni votre entrepôt, ni ce que veut dire « charger un mois ». Il lance une commande et regarde son code de sortie : c'est pourquoi le Cours 05 s'est terminé sur une porte d'entrée unique, au code de sortie honnête.

Analogie — le contremaître. Il ne visse rien lui-même. Il sait qui fait quoi, dans quel ordre, à quelle heure, et il note ce qui a été fait. Une consigne floue, il la transmet floue ; un travail mal fait, il ne le rattrape pas : il le consigne.

Airflow, l'orchestrateur de ce cours, est une petite équipe de services, et chacun peut tomber seul.

Le service	Son conteneur	Ce qu'il fait
le scheduler	`airflow-scheduler`	décide quoi lancer et quand ; avec l'exécuteur local, il lance aussi les tâches
le lecteur de DAG	`airflow-dag-processor`	relit le dossier des définitions et inscrit ce qu'il y trouve
le serveur d'API	`airflow-apiserver`	l'interface web, et l'API que les tâches appellent pour dire où elles en sont

© 2026 iAlexMG. Tous droits réservés.

Le service	Son conteneur	Ce qu'il fait
le déclencheur	`airflow-triggerer`	tient les attentes sans occuper de place : délais, capteurs différables
la base de métadonnées	`airflow-db`	l'état de tout ce qui précède, dans un PostgreSQL à elle

- Le lecteur de DAG est un service à part depuis Airflow 3 : une définition qui ne compile pas ne peut plus faire tomber le scheduler avec elle.

À retenir — la base de métadonnées n'est pas votre entrepôt. Elle ne contient rien de Boréal : que des états, des dates et des journaux. Un orchestrateur qui partagerait la base qu'il alimente tomberait avec elle, et ne pourrait même plus dire pourquoi.

2. Le DAG, et ce que ses flèches ne disent pas

DAG veut dire graphe acyclique dirigé : des tâches, des flèches entre elles, et aucun chemin qui revienne sur ses pas. Sans cette contrainte, il n'existerait pas d'ordre de passage.

Le DAG quotidien de Boréal se lit d'une ligne : `bronze` → `raw` → `corpus` → `dbt_build`.

Tâche	Ce qu'elle lance
`bronze`	`python -m pipeline bronze`, qui refait le Parquet de l'usine
`raw`	`python -m pipeline raw`, limité à la journée de l'exécution
`corpus`	`python -m pipeline corpus`, qui dépose les documents
`dbt_build`	rien, pour l'instant : la place que la formation Analytics Engineering remplira

Trois choses que ces flèches ne disent pas, et qu'on croit souvent qu'elles disent :

- le temps : une flèche veut dire « pas avant que », pas « juste après » ; entre deux tâches, il peut s'écouler une seconde ou une heure ;
- le passage de données : rien ne circule le long d'une flèche ; si `raw` a besoin de ce que `bronze` a écrit, c'est parce que les deux touchent le même entrepôt ;
- ce qui est hors du graphe : une tâche d'un autre DAG qui écrit le même fichier au même moment, le graphe ne la voit pas (section 7).

Piège — le DAG qui raconte l'ordre dans lequel on a écrit les tâches. Tout enchaîner en ligne rassure, mais une dépendance qui n'existe pas coûte du temps à chaque exécution et masque les vraies. Une flèche ne se pose que là où la seconde tâche échouerait, ou mentirait, si la première n'avait pas fini.

© 2026 iAlexMG. Tous droits réservés.

3. La planification, et l'intervalle de données

La seule idée d'Airflow qui se comprend mal, et l'erreur qu'elle cause coûte des données fausses plutôt qu'une panne visible. Une exécution planifiée porte deux temps sans rapport entre eux.

Le temps	Ce qu'il est
l'intervalle de données	la tranche que cette exécution doit traiter : du 15 mars au 15 mars
l'heure d'exécution	le moment où la machine a lancé la tâche : aujourd'hui, 16 h 23

- Le premier est une propriété des données, le second un accident de calendrier. Une exécution du 15 mars rejouée aujourd'hui traite toujours le 15 mars : c'est ce qui la rend rejouable.
- Une tâche écrit donc ses dates à partir de l'exécution, jamais de l'horloge. `{{ ds }}` est la date logique de l'exécution, en `AAAA-MM-JJ` : le début de l'intervalle pour une exécution planifiée, et la date demandée pour une exécution lancée à la main, dont Airflow calcule l'intervalle à partir de l'heure du lancement. Une commande lit donc la date logique.

```
python -m pipeline raw --depuis {{ ds }} --jusqua {{ ds }}
```

Piège — l'horloge dans la commande. `date +%F` dans un `bash_command`, `datetime.now()` dans une fonction de tâche : les deux donnent l'heure de la machine, pas l'intervalle de l'exécution. Le pipeline tourne, ne signale rien, et remplit le mauvais jour.

- La planification elle-même : un rythme, qui découpe le temps en tranches (`@daily`, `@monthly`), une date de début, et au besoin une date de fin. Une exécution part quand sa tranche est finie : celle du 15 mars à minuit le 16, celle de janvier le 1er février.

Rappel — Airflow 3 a changé le sens de `@daily`. Écrite seule, l'expression ne découpe plus de tranche : l'exécution de minuit porte la date du jour qui commence, et une commande qui lit `{{ ds }}` charge une journée encore vide, sans jamais échouer. Le DAG écrit donc son intervalle en toutes lettres : `schedule=CronDataIntervalTimetable("@daily", timezone="UTC")`. La documentation d'Airflow 2 suppose l'ancien sens.

Piège — la date de fin est inclusive, et l'exécution de trop est verte. Une fin posée au premier avril fait naître l'exécution d'avril en plus des trois voulues. Le chargeur ne trouve aucun fichier, rend zéro ligne, et l'étape passe à « réussi ». Le motif reviendra au Cours 11 : une exécution verte à zéro ligne ne se voit que si l'on a déclaré ce qu'on attendait. La fin se pose au dernier jour de la dernière tranche voulue.

4. Le rattrapage

Mettez en route un DAG mensuel dont la date de début remonte à trois mois : l'orchestrateur lance trois exécutions, une par intervalle manqué, dans l'ordre. C'est ce qui distingue un orchestrateur d'une tâche planifiée du système d'exploitation, qui n'a aucune mémoire de ce qu'elle a manqué.

- Le rattrapage ne vaut que si l'état obtenu après les trois exécutions égale celui d'un chargement des trois mois d'un seul bloc. Sinon, ce n'est pas une reprise, c'est une deuxième histoire.

© 2026 iAlexMG. Tous droits réservés.

- L'égalité exige que chaque tâche se limite à son intervalle. Le pipeline du Cours 05, qui charge « tout ce qu'il ne connaît pas encore », la casse sournoisement : la première exécution avale les trois mois, les deux suivantes ne trouvent rien, et l'état final est juste par hasard. Rejouez le deuxième mois seul, et rien ne vient. D'où la fenêtre que le labo ajoute : `--depuis` et `--jusqua`.
- Le rattrapage se refuse aussi : un DAG quotidien parti de quatre ans en arrière créerait des milliers d'exécutions. L'historique de Boréal est déjà chargé ; le quotidien part sans rattrapage, et c'est le mensuel qui rattrape.

À retenir — le rattrapage est une mesure, pas une promesse. Il se vérifie en chargeant les mêmes mois deux fois, par exécutions successives puis d'un bloc, et en comparant les tables ligne par ligne. C'est la preuve de fin du labo.

5. Rejouable : la fenêtre, le journal, et la reconstruction

Le Cours 03 a posé l'idempotence : relancer ne double rien. Le Cours 06 la pose une seconde fois, à l'échelle du DAG, et trois mécaniques distinctes y concourent.

La mécanique	Ce qu'elle garantit	Où elle vit
la fenêtre	l'exécution ne regarde que les lots de sa tranche	`--depuis` et `--jusqua`
le journal des chargements	un fichier déjà chargé sous la même empreinte ne se recharge pas	`pipeline.chargements`, Cours 03
la reconstruction	réécrire l'ensemble redonne le même ensemble	le `COPY ... OVERWRITE` du Cours 02

- La fenêtre et le journal ne se remplacent pas. La fenêtre travaille sur des noms de fichiers et écarte ce qui n'est pas de la tranche sans rien lire ; le journal travaille sur des empreintes et doit lire chaque fichier restant. Filtrer par la fenêtre d'abord, c'est ne pas relire 1 572 fichiers pour en charger 14.
- Deux fenêtres qui se chevauchent ne doublent rien : c'est le journal qui l'assure. La fenêtre décide du travail à examiner, le journal du travail à faire.
- Toutes les étapes ne se fenêtrent pas. Le bronze réécrit ses partitions et compare ses totaux ; le corpus compare des empreintes de documents, qui n'ont pas de date. Un DAG mélange des tâches incrémentales et des tâches qui reconstruisent tout, et savoir laquelle on écrit évite de recharger l'historique tous les matins, ou de croire l'avoir rechargé.

6. Ce qui rate, ce qui tarde, ce qui attend

Trois situations qui se ressemblent de loin, et trois réponses.

© 2026 iAlexMG. Tous droits réservés.

La situation	La réponse	Dans Airflow
une tâche rate	la nouvelle tentative, pour ce qui a une chance de passer au second coup : un verrou tenu, un service qui redémarre ; jamais une erreur de SQL	`retries`, `retry_delay`
une tâche tarde, sans rater	l'alerte de retard : un point de référence, un délai, et ce qu'on fait quand il est dépassé	les Deadline Alerts
une tâche attend une condition	le capteur, qui vérifie régulièrement ; différable, il rend sa place pendant qu'il attend	le déclencheur

- La tâche qui tarde est la plus coûteuse, parce que rien ne s'allume : un chargement de trois heures au lieu de dix minutes finit peut-être par passer, et personne ne le sait.
- La distinction entre erreur passagère et erreur définitive est celle du Cours 05, remontée d'un cran.

Rappel — les SLA d'Airflow 2 n'existent plus. Un SLA, un engagement de service, était un délai au-delà duquel une tâche en retard déclenchait une alerte. Airflow 3.0 les a retirés au profit des Deadline Alerts. Chercher « SLA airflow » mène à la documentation d'Airflow 2, et à du code mort.

7. Ce qui se partage se réserve

Deux DAG, chacun correct, lancent au même moment une tâche qui écrit l'entrepôt DuckDB. La seconde reçoit une erreur qui ne parle ni de DAG ni de tâche :

```
IO Error: Could not set lock on file "/projet/plateforme/entrepot/boreal.duckdb":
Conflicting lock is held in /usr/python/bin/python3.13 (PID 94).
```

- C'est la règle du Cours 03 qui remonte : un seul processus à la fois sur le fichier de l'entrepôt. Le nombre d'exécutions simultanées d'un DAG ne protège ce DAG que de lui-même.
- Airflow appelle pool un nom, un nombre de places, et chaque tâche qui le déclare attend son tour. Un pool d'une place, réclamé par toute tâche qui appelle le pipeline, sérialise les écritures sans changer une ligne du pipeline.

À retenir — un pool nomme une ressource, pas une tâche. Ce qu'on protège ici, c'est le fichier `boreal.duckdb`. Tout ce qui l'écrit prend le pool, quel que soit le DAG où il vit, et un lecteur du DAG comprend pourquoi en lisant le nom.

8. Le point de couture

Le DAG quotidien finit sur une tâche qui ne fait rien, `dbt_build`. Ce n'est pas un oubli.

© 2026 iAlexMG. Tous droits réservés.

- La chaîne de Boréal ne s'arrête pas à la `RAW`. La formation Analytics Engineering décode ce que le Cours 03 y a déposé et le livre en tables de faits et de dimensions, avec dbt, qui se lance par une commande, `dbt build`, comme le pipeline par `python -m pipeline`.
- Cette tâche vide est l'endroit, nommé, où les deux formations se rejoignent : la poser coûte une ligne ; ne pas la poser coûterait la refonte du DAG le jour où AE arrive.
- Un DAG d'entraînement, en Machine Learning Engineering, réemploie le même outil et les mêmes notions, intervalle de données compris ; seul change ce que les tâches font.

9. Un pipeline orchestré, en pratique

Un orchestrateur ne rend un pipeline sûr que si chaque promesse se vérifie. L'ingénieur de données pose toujours les mêmes questions.

Question	Ce qu'on vérifie	Chez Boréal (labo)
La tranche est-elle respectée ?	une journée chargée seule	1 fichier, 222 lignes (4.1)
Relancer double-t-il ?	la même tranche deux fois	« à jour », 0 fichier (4.2)
Une consigne fausse se voit-elle ?	les garde-fous	trois arrêts nets (4.3 à 4.5)
Le rattrapage tient-il sans orchestrateur ?	un test	`5 passed` (5.2)
Deux DAG se marchent-ils dessus ?	le pool de l'entrepôt	`entrepot`, une place (7.1)
Le DAG est-il bien lu ?	les erreurs d'importation	`No data found` (8.4)
Une journée rejouée traite-t-elle sa journée ?	la date logique	le 2026-03-15, un fichier de télémétrie (8.5, 8.6)
Le rattrapage égale-t-il le bloc ?	les tables, ligne par ligne	14 tables, 0 écart (10.8)
La plateforme reste-t-elle hors du dépôt ?	le `git status`	ni compose ni journaux d'Airflow (11)

Le Cours 11 fera de plusieurs de ces vérifications des contrats, jugés à chaque exécution.

À retenir

- Un orchestrateur décide quoi lancer, quand et dans quel ordre, et garde la trace. Il ne transforme rien : le travail reste au pipeline, derrière une porte d'entrée unique.

© 2026 iAlexMG. Tous droits réservés.

- Une flèche d'un DAG dit « pas avant que ». Elle ne dit ni le délai, ni le passage de données, et ne voit rien de ce qui se passe hors du graphe.
- L'intervalle de données n'est pas l'heure à laquelle l'exécution tourne. Une commande qui lit l'horloge remplit le mauvais jour sans jamais échouer ; elle lit la date logique.
- Le rattrapage crée une exécution par intervalle manqué. Il ne vaut que si chaque tâche se limite à son intervalle, et cela se vérifie contre un chargement d'un bloc.
- La fenêtre choisit quoi regarder, le journal décide quoi charger, la reconstruction réécrit tout.
- Une tâche qui rate se réessaie, une tâche qui tarde se signale, une tâche qui attend se diffère.
- Une ressource partagée entre DAG se réserve par un pool, qui nomme la ressource et non la tâche.
- Une tâche vide et nommée vaut mieux qu'un DAG à refaire : `dbt_build` est le point de couture avec Analytics Engineering.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_06_DE.md`. Faites-les après le labo : plusieurs énoncés supposent que les deux DAG existent, qu'ils ont tourné au moins une fois, et que la comparaison du rattrapage a été faite.

Annexe A — Le détail des étapes du labo

A.0 Avant de commencer : construire et démarrer la pile

Notions : section 1, les services d'Airflow.

- `wsl` ouvre Ubuntu dans le dossier courant : la fenêtre 2 s'ouvre donc dans `plateforme`, où `docker compose` trouve son fichier et le `.env` voisin.
- `docker compose build airflow-init` construit l'image `boreal-airflow`, une fois : les cinq services d'Airflow la partagent. Le téléchargement de l'image officielle pèse 3,18 Go et ne se refait jamais ; chez l'auteur, la construction a pris un peu plus de deux minutes, et le premier démarrage, avec le téléchargement de la base de métadonnées (`postgres:16`), moins d'une minute.
- `docker compose up -d` nomme les quatre services qu'il démarre. Le compose de la plateforme décrit aussi ceux des cours suivants (la base de séries du Cours 08, MQTT et Kafka du Cours 09, Grafana du Cours 12) : un `up -d` sans nom les démarrerait tous, et téléchargerait leurs images.

© 2026 iAlexMG. Tous droits réservés.

- La base de métadonnées démarre d'abord ; `airflow-init` passe une seule fois : il crée les tables d'Airflow, le compte `airflow` et le pool de l'entrepôt, puis s'arrête. Les quatre autres attendent qu'il ait fini.
- La pile mesurée demande un peu plus de 1,1 Go de mémoire, là où la documentation d'Airflow demande d'accorder 4 Go à Docker pour sa pile complète.

A.1 Étape 1 : régénérer les sources

Labo : `Labo_06_DE.md`, étape 1.

Notions : section 3, de nouvelles tranches arrivent.

- 1.1. Le générateur vit dans `_univers`, à côté du projet : `..\_univers\…` le trouve depuis `Projet_Boreal_intermediaire`, et `--projet .` désigne le dossier courant. Sans option, le générateur s'arrête au 31 décembre 2025 : c'est l'état des Cours 01 à 05. `--jusqua 2026-03` produit l'état de Boréal au 31 mars 2026, trois mois plus tard. Il réécrit `sources/` en entier, et les mois déjà livrés reviennent identiques au bit près : le générateur est déterministe, et c'est ce qui rend le labo rejouable.
- Le bilan change : 51 extraits mensuels de l'ERP, 27 185 commandes ; l'historien de l'usine passe de 92 à 182 jours ; 974 jours de télémétrie ; 34 documents au catalogue. L'entrepôt, lui, n'a pas bougé : il en est resté à ce que le Cours 05 y a chargé.

A.2 Étape 2 : la fenêtre

Labo : `Labo_06_DE.md`, étape 2.

Notions : section 3, l'intervalle de données ; section 5, la fenêtre.

- 2.1. La fenêtre répond à une seule question : « ce lot tombe-t-il dans ma tranche ? ». Un lot est ce que le Cours 03 écrit dans la colonne `_lot` : le mois d'un extrait, le jour d'une archive de télémétrie, ou « complet ». Elle ne touche ni l'entrepôt ni les fichiers, d'où un module à elle.

La décision inscrite dans `fenetre.py`	**Pourquoi**
une borne de mois vaut tout le mois	`--jusqua 2026-01` ne doit pas s'arrêter au premier janvier
« complet » et l'inconnu passent	une tranche de temps ne fait pas disparaître une livraison en silence
une borne illisible lève `DateIllisible`	mieux vaut un arrêt net qu'un filtre qui écarte tout sans le dire

A.3 Étape 3 : brancher la fenêtre et les livraisons

Labo : `Labo_06_DE.md`, étape 3.

Notions : section 5, la fenêtre et le journal ; section 7, ce qui se partage.

© 2026 iAlexMG. Tous droits réservés.

- 3.1. `raw.py` gagne la fenêtre et les livraisons. `a_charger` applique deux filtres, dans cet ordre : la fenêtre, sur les noms, puis le journal, sur les empreintes. `sources_de` rend les livraisons demandées par leur préfixe (« erp », « telemetrie ») : le DAG mensuel prend l'ERP et les garanties, le quotidien l'usine, la télémétrie, la météo et le corpus, et les deux ne se disputent rien. Un préfixe inconnu lève `LivraisonInconnue`, une erreur du domaine, jamais un `SystemExit`, qui traverserait l'encadrement de l'étape sans fermer sa ligne.
- 3.2 à 3.5. Les quatre autres étapes reçoivent les deux arguments et n'en font rien : la porte d'entrée les passe à toutes, et chacune dit dans sa docstring pourquoi la fenêtre ne s'applique pas chez elle. `bronze.py` reçoit la connexion sans valeur par défaut, parce qu'il s'en sert ; les trois autres gardent `con=None`.
- 3.6. La porte d'entrée gagne trois options, `--depuis`, `--jusqua` et `--livraisons`, et une colonne à `ETAPES` : celle qui dit si une fenêtre restreint l'étape. Elle traduit les bornes en `Fenetre`, refuse une borne illisible ou deux bornes inversées avant d'ouvrir quoi que ce soit, et avertit quand une étape qui reconstruit tout reçoit une fenêtre : un DAG quotidien lance `bronze` et `raw` dans la même commande, et seule la seconde se restreint.
- 3.7. Seule `raw` travaille livraison par livraison : elle seule peut se limiter à une tranche.

A.4 Étape 4 : une journée, et les garde-fous

Labo : `Labo_06_DE.md`, étape 4.

Notions : section 5, la fenêtre et le journal.

- 4.1. Sur 1 989 fichiers, la fenêtre en retient un, et le journal confirme qu'il est nouveau : un fichier, 222 messages, un dixième de seconde.
- 4.2. C'est le journal qui parle ici, pas la fenêtre : le fichier est dans la tranche, mais son empreinte est déjà connue.
- 4.3. Une livraison inconnue fait échouer l'étape : le pipeline l'inscrit et rend 1, et le message donne la liste des livraisons connues.
- 4.4 et 4.5. Une date illisible et une fenêtre vide arrêtent la porte d'entrée avant tout travail, sur un message qui dit quoi corriger.

A.5 Étape 5 : deux tests de plus

Labo : `Labo_06_DE.md`, étape 5.

Notions : section 4, le rattrapage ; section 5, le chevauchement.

- 5.1. Le premier test vérifie la fenêtre sans base ni fichier : un mois, un jour, une relivraison (`2023-03_bis` reste mars 2023), un export complet, une borne illisible. Le second vérifie la promesse du rattrapage : trois fenêtres d'un mois chargent ce qu'une seule aurait chargé, une fenêtre ouverte ne trouve plus rien, et deux fenêtres qui se chevauchent ne doublent rien.
- 5.2. Cinq tests, en 0,4 s chez l'auteur, sans les 177 Mo de Boréal et sans conteneur. Le jour où un DAG se comportera mal, on saura que le pipeline, lui, tient.

© 2026 iAlexMG. Tous droits réservés.

A.6 Étape 6 : la pile et ses fichiers

Labo : `Labo_06_DE.md`, étape 6.

Notions : section 1, ce qu'un orchestrateur fait, et ses services.

- 6.1. `up -d` sur des services qui tournent ne fait rien ; il rallume ceux qui se sont arrêtés, avec leurs données. `restart: unless-stopped` les fait repartir quand Ubuntu redémarre.
- 6.2. La liste des DAG reste vide tant que le dossier où Airflow les cherche n'existe pas.
- 6.3. Ce dossier est dans votre dépôt : le compose dit à Airflow de lire `/projet/01_de/dags`, à travers le montage du projet. Aucun montage propre aux DAG : Docker créerait sinon le dossier avant vous.
- 6.4. La recette de l'image, en trois gestes : partir de l'image officielle d'Airflow 3.3.1 ; ajouter, en `root`, le pilote ODBC de PostgreSQL, qui servira au Cours 12 ; ajouter, en `airflow`, DuckDB 1.5.5 et `pyodbc`. DuckDB y est à la même version que dans l'image du pipeline du Cours 05, et ce n'est pas un détail : les deux écrivent le même fichier d'entrepôt. Les dépendances se posent à la construction, pas au démarrage : l'installation au lancement, que l'image officielle permet par une variable, se referait à chaque démarrage, et la documentation la réserve aux essais.
- 6.5. Ce que porte la définition que les services d'Airflow partagent, écrite une fois sous l'ancre `&airflow-commun` :

Ce que la définition partagée porte	**Ce que ça veut dire**
`AIRFLOW__CORE__EXECUTOR: LocalExecutor`	les tâches s'exécutent dans des processus du scheduler, sur cette machine ; ni Redis, ni ouvrier Celery
`AIRFLOW__DATABASE__SQL_ALCHEMY_CONN`	la base de métadonnées, un PostgreSQL à elle, distinct de celui de l'ERP
`AIRFLOW__CORE__DAGS_FOLDER: /projet/01_de/dags`	Airflow lit vos DAG dans votre dépôt
`AIRFLOW__CORE__LOAD_EXAMPLES: 'false'`	l'interface reste vide des dizaines d'exemples livrés avec Airflow
`PIPELINE_PROJET: /projet` et `PYTHONPATH: /projet/01_de`	le paquet `pipeline` est importable dans le conteneur, et il y trouve le projet
`..:/projet`	tout `Projet_Boreal_intermediaire` est monté : les sources, le lac, l'entrepôt et votre dépôt
`./airflow/logs` et `./airflow/config`	les journaux et la configuration d'Airflow, du côté de la plateforme

- Les services, ensuite : `airflow-db`, la base de métadonnées ; `airflow-init`, qui passe une fois ; `airflow-apiserver`, l'interface et l'API sur le port 8080 ; `airflow-scheduler`, qui décide et exécute ; `airflow-dag-processor`, qui relit les DAG ; `airflow-triggerer`, qui tient les attentes. Cinq conteneurs et non sept : le compose officiel ajoute Redis et un ouvrier Celery, qui servent à répartir les tâches sur plusieurs machines.

© 2026 iAlexMG. Tous droits réservés.

A.7 Étape 7 : le pool de l'entrepôt

Labo : `Labo_06_DE.md`, étape 7.

Notions : section 7, ce qui se partage se réserve.

- 7.1. `airflow-init` a posé le pool `entrepot`, une place, au premier démarrage. Sans lui, relevé sur le poste de l'auteur en laissant les deux DAG démarrer ensemble, la seconde tâche meurt sur `Could not set lock`, repart cinq minutes plus tard, et le DAG affiche un rouge dont la cause n'est pas dans son graphe. Une dépendance ordonne deux tâches d'un même graphe ; un pool borne le nombre de tâches qui touchent une ressource, à travers tous les DAG : c'est le seul outil d'Airflow qui voit plus loin qu'un graphe.

A.8 Étape 8 : le DAG quotidien

Labo : `Labo_06_DE.md`, étape 8.

Notions : section 2, le DAG ; section 3, l'intervalle ; section 6, ce qui rate et ce qui tarde ; section 8, le point de couture.

- 8.1. `prevenir_retard` écrit le dépassement dans le journal ; en production, elle enverrait un courriel. Une fonction ordinaire s'enveloppe dans `SyncCallback` : `AsyncCallback` attend une coroutine, et lui donner une fonction fait échouer la lecture du DAG.
- 8.2. Ce que fait chaque réglage du DAG quotidien :
  - `CronDataIntervalTimetable("@daily", timezone="UTC")` : l'intervalle écrit en toutes lettres ;
  - `catchup=False` : l'historique a été chargé d'un bloc, personne ne veut 1 500 exécutions ;
  - `max_active_runs=1` et `retries=1`, cinq minutes d'attente : une écriture refusée parce qu'un autre processus tenait l'entrepôt passe au second essai, une erreur de SQL non ;
  - `DeadlineAlert`, 30 minutes après la mise en file : l'alerte de retard ;
  - chaque tâche prend le pool `entrepot` ; `raw` reçoit `{{ ds }}` et ses livraisons ; `dbt_build` est une tâche vide.
- 8.3. Le lecteur de DAG relit toutes les 30 secondes les fichiers qu'il connaît, mais ne cherche les nouveaux que toutes les 5 minutes. Un DAG retouché peut apparaître sur plusieurs lignes : une par version, chaque changement de structure en crée une.
- 8.5. `--logical-date` fixe la journée traitée, le 15 mars, quelle que soit l'heure du lancement. L'exécution se nomme d'après l'heure du lancement (`manual__` suivi de cette heure) : c'est sa date logique qui porte le 15 mars.
- 8.6. Sans rattrapage, le scheduler crée seul la dernière journée finie, la veille, dès que le DAG est inscrit : elle passe la première, et celle du 15 mars attend son tour (`max_active_runs=1`). Ce que les tâches écrivent, chez l'auteur :

© 2026 iAlexMG. Tous droits réservés.

Tâche	La veille, créée seule	Le 2026-03-15
`bronze`	754 572 lignes réécrites en 33,8 s, fenêtre ignorée	la même chose, rien n'ayant changé
`raw`	859 lignes : les deux exports livrés en entier qui ont changé à la régénération, 825 bons de travail et 34 documents au catalogue, puisque « complet » entre dans toutes les fenêtres	949 lignes : l'archive de télémétrie du jour
`corpus`	3 documents déposés	rien : la veille les a déposés
`dbt_build`	rien, et c'est voulu	rien

- Les 34 secondes du bronze : le conteneur écrit sur des fichiers de Windows, à travers la frontière de WSL 2, le coût mesuré au Cours 05. Relancer la même journée plante sur `UniqueViolation` : Airflow n'exécute une date qu'une fois par DAG (dépannage du labo).

A.9 Étape 9 : le témoin et le DAG mensuel

Labo : `Labo_06_DE.md`, étape 9.

Notions : section 3, l'intervalle mensuel ; section 4, le rattrapage.

- 9.1. Le témoin se copie avant que le DAG mensuel existe : écrit plus tôt, le DAG commencerait à charger pendant la copie. Il se copie aussi après les deux exécutions quotidiennes. Copié pendant que celle du 15 mars attendait son tour, un témoin a manqué ses 949 lignes de télémétrie, et la copie a réussi sans rien dire. La comparaison de l'étape 10 ne le verrait pas : elle ne regarde que les quatorze tables du DAG mensuel.
- 9.2. `PIPELINE_CHEMINS_ENTREPOT` détourne le pipeline vers un autre fichier sans toucher au `config.toml` : la surcharge du Cours 05. Oubliée, la variable ferait écrire tout le reste du labo dans le témoin, et la comparaison comparerait un fichier avec lui-même.
- 9.3. Ce que fait chaque réglage du DAG mensuel :
  - `MOIS` lit le mois dans la date logique, en `AAAA-MM` ; une borne de mois vaut tout le mois ;
  - `end_date` est inclusive : posée au 1er avril, elle ferait naître une quatrième exécution, verte et vide (section 3) ;
  - `catchup=True` : il rattrape ;
  - une seule tâche : le mois n'apporte que des fichiers à charger, le bronze de l'usine est quotidien et le corpus n'a pas de rythme.
- 9.4. `run_after` est le moment où l'exécution a le droit de partir, une fois son mois fini, et il la nomme ; `logical_date` est le début du mois traité. L'exécution de janvier s'appelle donc `scheduled__2026-02-01`. Chaque exécution charge quatorze fichiers, chez l'auteur :

© 2026 iAlexMG. Tous droits réservés.

Mois	Fichiers	Lignes
2026-01	14	6 452
2026-02	14	6 506
2026-03	14	7 706
Total	42	20 664

- Les mêmes totaux que le témoin : encourageant, pas une preuve. Deux comptes égaux ne disent rien du contenu.

A.10 Étape 10 : la preuve

Labo : `Labo_06_DE.md`, étape 10.

Notions : section 4, le rattrapage est une mesure.

- 10.2 à 10.4. Chaque entrepôt s'attache sous son nom, en lecture seule. La comparaison exclut la seule colonne qui doit différer, `_charge_le`, l'heure du chargement, et se fait dans les deux sens.
- 10.5 et 10.6. Les deux fichiers se relâchent avant le script, qui les ouvre à son tour.
- 10.7. Un nom de table n'est pas une valeur : il ne se calcule pas dans une requête, et le SQL ne sait pas boucler sur une liste de tables. Quand un contrôle doit passer sur une liste d'objets, il sort du SQL et rentre dans le code, qui lit la liste dans `duckdb_tables()`. Ce que fait le script :
    - `EXCEPT ALL`, et non `EXCEPT` : le premier compte les exemplaires, le second compare des ensembles et écarte les doublons, si bien qu'un mois chargé deux fois lui serait invisible ;
    - l'écart des comptes entre dans le verdict : une preuve qui ne peut pas échouer ne prouve rien.
- 10.8. Trois exécutions successives du DAG mensuel ont produit exactement ce qu'un chargement d'un bloc aurait produit. Cette preuve autorise à rejouer un mois seul, à remettre un DAG en route après une panne de plusieurs jours, et à reconstruire l'entrepôt en laissant le rattrapage travailler une nuit. Sans elle, chacune de ces opérations serait un pari.

A.11 Étape 11 : versionner

Labo : `Labo_06_DE.md`, étape 11.

Notions : aucune section de ce cours ; le dépôt et son `.gitignore` viennent du Cours 01, annexe A.1.

© 2026 iAlexMG. Tous droits réservés.

- `git status` ne montre que ce qui est à vous : les DAG, la fenêtre, les modules retouchés, les tests et le script de comparaison. Le `docker-compose.yml`, le `Dockerfile.airflow` et les journaux d'Airflow vivent dans `plateforme`, hors du dépôt : en entreprise, ils sont dans le dépôt de l'équipe qui exploite la plateforme.
- `docker compose stop` éteint la pile sans rien perdre : la base de métadonnées garde l'historique des exécutions.

En entreprise, les mêmes gestes changent d'outil :

Geste du labo	**En entreprise**
Airflow sur le poste (6)	un Airflow géré : Amazon MWAA, Google Cloud Composer, Astronomer ; ou l'orchestrateur de la plateforme de données, les tâches planifiées de Databricks ou de Snowflake
Les DAG dans le dépôt (8, 11)	déployés par l'intégration continue vers le dossier des DAG de l'instance
Le pool de l'entrepôt (7)	les mêmes pools, ou une file par ressource partagée
L'alerte de retard (8)	branchée sur un canal d'astreinte : courriel, messagerie, outil de garde
La preuve du rattrapage (10)	le même contrôle, automatisé après chaque rattrapage par des tests de données (Cours 11)

© 2026 iAlexMG. Tous droits réservés.